

Chapter 2: Document Indexing, Storage, and Compression

2.1 Inverted Index Construction

An **Inverted Index** is the most important data structure used in Information Retrieval (IR) systems.

It stores a mapping from **terms (keywords)** to the **list of documents** in which those terms occur.

This structure allows the system to quickly find all documents that contain a specific term — making searching extremely fast and efficient.

Example:

If we have 3 documents:

- Doc1: “Information retrieval is useful”
- Doc2: “Retrieval systems use index files”
- Doc3: “Indexing improves information access”

Then the inverted index will look like this:

Term	Documents
information	Doc1, Doc3
retrieval	Doc1, Doc2
index	Doc2
indexing	Doc3
useful	Doc1

This structure helps the IR system find relevant documents immediately when a user types a query.

Main Components of an Inverted Index:

1. **Dictionary (Vocabulary):**
 - Contains all **unique terms** appearing in the collection.
 - Each term has a pointer to its postings list.
 - Example: {information, retrieval, system, index, compression...}
2. **Postings List:**
 - A list of **document IDs (docIDs)** where the term occurs.
 - May also store **term frequency** and **positions** of the term in the document.
 - Example:
 - $retrieval \rightarrow [(Doc1, 2), (Doc2, 1)]$

Together, these two components make the inverted index.

Steps in Inverted Index Construction:

1. **Document Collection:**
 - Start with a set of text documents to be indexed.
2. **Tokenization:**
 - Split each document into **tokens (words)** using spaces or punctuation marks.
 - Example: “Information Retrieval System” → [Information, Retrieval, System].
3. **Normalization:**
 - Convert words to lowercase, remove punctuation, and perform stemming/lemmatization.
 - Example: “Running”, “Runs”, “Ran” → “Run”.
4. **Stop Word Removal:**
 - Remove common words like *the, is, of, and, a* which do not affect meaning.
5. **Term–Document Pair Creation:**
 - For each word, record the document in which it appears.
 - Example: (information, Doc1), (retrieval, Doc1), (index, Doc2).
6. **Sorting and Merging:**
 - Sort the pairs by term and merge all pairs of the same term to form the postings list.
7. **Index Storage:**
 - Store the dictionary and postings list efficiently for fast access and updates.

Example (Step by Step):

Documents:

- D1: “Data retrieval system”
- D2: “System uses data index”

Step 1 – Tokenization:

D1 → [data, retrieval, system]

D2 → [system, uses, data, index]

Step 2 – Term-Document Pairs:

(data, D1), (retrieval, D1), (system, D1), (system, D2), (uses, D2), (data, D2), (index, D2)

Step 3 – Sorting:

(data, D1), (data, D2), (index, D2), (retrieval, D1), (system, D1), (system, D2), (uses, D2)

Step 4 – Merging:

- data → [D1, D2]
- index → [D2]
- retrieval → [D1]
- system → [D1, D2]
- uses → [D2]

Advantages of Inverted Index:

1. Enables **fast retrieval** of documents for a given term.
2. Reduces **search time complexity** compared to scanning the entire collection.

3. Supports **ranking and relevance calculation** using term weights.
4. Can be **compressed** to save storage space.
5. Allows **incremental updates** when new documents are added.

2.2 Compression Techniques

Definition:

Compression is the process of reducing the size of data files (like indexes or postings lists) by encoding them efficiently.

In Information Retrieval (IR) systems, compression helps in storing large indexes in less space and retrieving data faster.

Need for Compression:

1. **Reduces storage space:** Indexes can be very large; compression saves disk memory.
2. **Faster retrieval:** Smaller files load quickly, improving search performance.
3. **Lowers I/O cost:** Less data is read from disk per query.
4. **Efficient memory usage:** Helps store large indexes in limited RAM.
5. **Network efficiency:** Compressed files can be transferred faster over networks.

Types of Compression in IR:

There are two major types of compression techniques used in IR systems:

(A) Dictionary Compression

(B) Postings List Compression

(A) Dictionary Compression:

The **dictionary** contains all the unique words (terms) in the document collection. To store it efficiently, the following methods are used:

1. **Front Coding:**
 - Similar words often share a common prefix.
 - Store the prefix once and keep only the different suffixes.
 - Example:
 - compute
 - *r (computer)
 - *ing (computing)
 - Saves space by removing repeated beginnings of words.
2. **Blocked Storage:**
 - Terms are divided into small blocks (like 8–16 terms per block).
 - The common prefix of the block is stored once, followed by suffixes.
 - Allows partial decompression and faster lookup.
3. **Hashing / Term Truncation:**
 - Store only the first few characters (e.g., first 8 letters) or a hash value of each word.

- Reduces storage but may lead to *collisions* (different words having same hash).

(B) Postings List Compression:

The **postings list** is the largest part of the index.

It stores document IDs, term frequencies, and sometimes positions of terms.

Compression here gives the most space savings.

Techniques:

1. Gap Encoding (Delta Encoding):

- Instead of storing full document IDs, store the difference (gap) between consecutive IDs.
- Example:
Document IDs: [3, 5, 8, 12]
Gaps: [3, 2, 3, 4]
- Smaller numbers are easier to compress.

2. Variable Byte (VB) Encoding:

- Uses fewer bytes for smaller numbers.
- Each integer is divided into 7-bit groups, with the 8th bit used as a continuation flag.
- Compact and simple to decode.

3. Gamma Coding:

- A binary-based method that encodes small integers efficiently.
- Best suited for encoding small gap values.
- Combines unary and binary codes.

4. Elias Delta / Golomb Coding (Advanced):

- Used in large-scale systems.
- Encodes numbers based on probability distribution for high efficiency.

Advantages of Compression:

1. Saves disk space and memory.
2. Improves retrieval speed and efficiency.
3. Reduces data transfer and I/O time.
4. Enables scalability for large document collections.
5. Cost-effective in terms of storage hardware.

2.3 Document Representation

Definition:

Document Representation refers to the method of expressing a text document in a form that a computer can understand and process for searching, matching, and ranking.

It converts unstructured textual data into a **structured numerical form** that can be compared with user queries.

Need for Document Representation:

1. Computers cannot understand natural language directly.
2. For efficient searching and retrieval, documents must be converted into a mathematical or logical model.
3. Representation helps in calculating similarity between **documents and queries**.
4. It supports ranking, clustering, and classification in IR systems.

Common Models of Document Representation:

The main models used are:

1. **Boolean Model**
2. **Vector Space Model (VSM)**
3. **Probabilistic Model**

1. Boolean Model:

- The simplest model based on **set theory** and **Boolean logic** (AND, OR, NOT).
- Each document is represented as a **set of terms** (keywords).
- A term is either **present (1)** or **absent (0)** in the document — no partial relevance.

Example:

Let's say

- Document D1 = {data, retrieval, system}
- Document D2 = {information, retrieval, index}

Query: *retrieval AND data*

→ Only D1 is retrieved because both terms appear there.

Advantages:

- Simple and easy to implement.
- Works well for small document sets.

Disadvantages:

- No ranking or partial matching.
- Treats all terms equally — ignores frequency and importance.

2. Vector Space Model (VSM):

- The most widely used model in modern IR systems.
- Each document is represented as a **vector of term weights** in a multi-dimensional space.
- Each dimension corresponds to a term from the dictionary.
- The **angle (cosine similarity)** between the query vector and document vector determines relevance.

Example:

Term	Doc1	Doc2	Query
information	1	0	1
retrieval	1	1	1
index	0	1	0

Cosine similarity between query and documents gives the most relevant result.

Advantages:

- Allows **partial matching** and **ranking**.
- Considers **term frequency** and **importance**.
- Works well for large text collections.

Disadvantages:

- Ignores word order and syntax.
- High dimensionality (many terms = large vectors).

3. Probabilistic Model:

- Based on estimating the **probability** that a document is relevant to a given query.
- Documents are ranked according to their **relevance probability**.
- Uses feedback from users to improve accuracy.

Formula (simplified):

$P(\text{Rel} \mid Q, D)$ = Probability that document D is relevant given query Q.

Advantages:

- Considers uncertainty and user feedback.
- Produces better ranking with large data.

Disadvantages:

- More complex than VSM.
- Requires training data and probability estimation.

Advantages of Proper Document Representation:

1. Enables efficient searching and ranking.
2. Reduces ambiguity by using structured data.
3. Helps in clustering and classification of documents.
4. Improves accuracy of IR systems.
5. Supports advanced features like TF-IDF weighting and cosine similarity.

2.4 Term Weighting

Definition:

Term Weighting refers to the process of assigning importance (weight) to each term (word) in a document.

The weight indicates **how important a term is** for identifying the document's relevance to a query.

Need for Term Weighting:

- Not all words are equally important.
- Common words (like “the”, “is”) appear in every document → low importance.
- Unique or frequent words in a particular document → high importance.
- Term weighting improves the **accuracy and ranking** of retrieved documents.

Objectives of Term Weighting:

1. To distinguish **important terms** from common ones.
2. To improve **retrieval performance** and **ranking**.
3. To reflect both **local importance** (within document) and **global importance** (across all documents).
4. To represent documents effectively in **Vector Space Models**.

Types of Term Weighting:

Term weights are generally calculated using three main components:

Component	Meaning	Based On
Term Frequency (TF)	Importance of a term within a document	Local
Inverse Document Frequency (IDF)	Rarity of a term across all documents	Global
Normalization	Adjusts for document length	Scaling

1. Term Frequency (TF):

- The number of times a term appears in a document.
- The higher the frequency, the more important the term is.

Formula:

$$TF_{t,d} = \text{Number of times term } t \text{ occurs in document } d$$

Example:

If the term “retrieval” appears 4 times in Document D1, then
 $TF(\text{retrieval}, D1) = 4$

Problem: Long documents naturally have higher TF — so normalization is needed.

2. Inverse Document Frequency (IDF):

- Measures how **unique or rare** a term is across all documents.
- Common terms (like “data”, “system”) appear in many documents → low IDF.
- Rare terms have high IDF → more useful for distinguishing documents.

Formula:

$$IDF_t = \log \left(\frac{N}{DF_t} \right)$$

Where:

- N = total number of documents
- DF_t = number of documents containing term t

Example:

If “retrieval” appears in 10 out of 1000 documents,
 $IDF(\text{retrieval}) = \log(1000/10) = \log(100) = 2$

3. TF-IDF Weighting:

- Combines **TF** and **IDF** for better accuracy.
- High weight if term appears **frequently in one document** but **rarely in others**.

Formula:

$$w_{t,d} = TF_{t,d} \times IDF_t$$

Example:

If $TF = 4$ and $IDF = 2 \rightarrow$
Weight = 8

Advantages:

- ✓ Simple and effective.
- ✓ Reduces the effect of common words.
- ✓ Improves ranking accuracy.

4. Normalization:

- Used to avoid bias towards long documents (which may have more words).
- Ensures fair comparison between short and long documents.

Formula (Cosine Normalization):

$$\text{Normalized Weight} = \frac{w_{t,d}}{\sqrt{\sum (w_{t,d})^2}}$$

Example Table:

Term	TF (D1)	DF	IDF	TF×IDF (Weight)
retrieval	4	10	2	8
data	10	500	0.3	3
system	5	300	0.52	2.6

→ Hence, **retrieval** is more important for D1 than “data” or “system”.

Advantages of Term Weighting:

1. Improves **accuracy and ranking** of search results.
2. Helps in **relevance measurement** between query and documents.
3. Makes IR systems **scalable and efficient**.
4. Reduces noise from common terms.
5. Supports **clustering, classification, and similarity measures**.

2.5 Storage and Retrieval of Indexed Documents.

Definition:

Once documents are **indexed** using techniques such as inverted indexing, the next step is to **store** these indexes efficiently and **retrieve** documents quickly based on user queries. This involves methods for **organizing, storing, and accessing** the index files and document data to achieve **fast and accurate information retrieval**.

Need for Efficient Storage and Retrieval:

1. The volume of documents on the web or digital libraries is huge.
2. IR systems must store indexes in a way that supports **quick searching and updating**.
3. Retrieval should return **relevant results** within milliseconds, even for millions of documents.

1. Storage of Indexed Documents

Efficient storage ensures that indexes and document data occupy minimal space while remaining easily accessible.

a. Index Storage Structure

The **inverted index** (term → list of document IDs) is the core structure used for storage.

Example:

Term	Posting List (Document IDs)
data	D1, D3, D5
retrieval	D1, D4
system	D2, D3, D6

This is stored in two main parts:

1. **Dictionary (Lexicon):**
Contains all unique terms and pointers to their posting lists.
2. **Postings Lists:**
For each term, stores:
 - Document IDs
 - Term frequency (TF)
 - Position of terms (for phrase search)

b. Storage Files Used

IR systems usually store index information in multiple files for performance optimization:

File Type	Description
Dictionary File	Stores unique terms and metadata.
Posting File	Contains posting lists (Doc IDs, TFs, positions).
Document File	Stores actual document content or pointers to full text.
Lexicon File	Maintains term ordering for faster lookup.

c. Compression in Storage

To save space and speed up access:

- Posting lists are **compressed** using techniques like:
 - Variable byte encoding
 - Gamma encoding
 - Delta encoding
- Reduces disk usage and improves caching efficiency.

2. Retrieval of Indexed Documents

Retrieval is the process of **matching a user's query** with stored indexes to find relevant documents.

a. Steps in Retrieval Process:

1. **Query Processing:**
 - User enters a query (e.g., “data retrieval system”).
 - Query is tokenized and preprocessed (stop-word removal, stemming).
2. **Term Lookup:**
 - Each query term is searched in the **dictionary**.
 - System retrieves corresponding **posting lists**.
3. **Merging Posting Lists:**
 - If multiple terms occur, their posting lists are **merged**.
 - Boolean or ranked retrieval models decide which documents to include.
4. **Ranking and Scoring:**
 - Documents are scored using **term weighting (TF-IDF)** or **similarity measures (cosine similarity)**.
5. **Result Display:**
 - Documents with highest scores are presented first.
 - Includes title, snippet, and link (like Google search results).

b. Retrieval Models Used

Model	Description	Example
Boolean Model	Uses logical operators (AND, OR, NOT)	“retrieval AND system”
Vector Space Model	Uses TF-IDF weighting and cosine similarity	Measures closeness between query and documents
Probabilistic Model	Estimates probability of document relevance	Okapi BM25

c. Ranking Example (Simplified):

Query: “*information retrieval system*”

Document	Score (TF-IDF Similarity)
D1	0.95
D2	0.82
D3	0.40

→ **D1** is displayed first because it's most relevant.

3. Updating and Maintenance of Indexes

IR systems must also handle document updates efficiently.

Operation	Description
Addition	New document → Add to index without rebuilding entire structure.
Deletion	Mark document as deleted or remove ID from posting list.
Modification	Update term frequency or re-index changed content.

4. Performance Considerations

Factor	Optimization Technique
Storage Space	Compression and efficient encoding
Search Speed	Use of memory caching and B-trees
Scalability	Distributed indexing and parallel retrieval
Reliability	Index backups and replication

5. Example: Google Search (Simplified)

1. Google crawls and indexes billions of pages (index stored in data centers).
2. When a user searches “IR systems”, the engine retrieves pre-indexed data.
3. Uses ranking algorithms (PageRank + TF-IDF-like measures).
4. Displays top-ranked results instantly.

Advantages:

- ✓ Faster access and retrieval time.
- ✓ Efficient use of disk space via compression.
- ✓ Supports real-time updates and scalability.
- ✓ Enables ranked, precise results.
- ✓ Easy to integrate with multiple retrieval models.